

Prefix Sum

1. What is Prefix Sum?

Prefix Sum is a technique used to store the **running sum of an array**.

Instead of calculating the sum again and again for the same part of an array, we calculate it once and store it.

This is especially useful when a problem asks for:

- sum from index `0` to `i`
 - sum of a range `l` to `r`
 - multiple range-sum queries
 - subarray sum problems
 - finding/counting subarrays with a particular sum
-

2. Basic Example

Suppose the array is:

```
arr = [2, 4, 6, 8, 10]
```

Indexes start from `0`:

Index:	0	1	2	3	4
Value:	2	4	6	8	10

There are two common ways to define a prefix-sum array.

3. Concept 1: Prefix Sum Including `i`

Here:

```
prefix[i] = arr[0] + arr[1] + ... + arr[i]
```

In simple words:

`prefix[i]` contains the sum **including the current index `i`**.

Example

```
arr = [2, 4, 6, 8, 10]
```

The prefix array becomes:

```
prefix = [2, 6, 12, 20, 30]
```

Because:

```
prefix[0] = 2
prefix[1] = 2 + 4 = 6
prefix[2] = 2 + 4 + 6 = 12
prefix[3] = 2 + 4 + 6 + 8 = 20
prefix[4] = 2 + 4 + 6 + 8 + 10 = 30
```

So:

```
prefix[2] = 12
```

because index `2` is also included.

Formula

```
prefix[i] = prefix[i - 1] + arr[i]
```

For the first element:

```
prefix[0] = arr[0]
```

Python

```
arr = [2, 4, 6, 8, 10]

prefix = [0] * len(arr)

prefix[0] = arr[0]

for i in range(1, len(arr)):
    prefix[i] = prefix[i - 1] + arr[i]

print(prefix)
```

Output:

```
[2, 6, 12, 20, 30]
```

4. Concept 2: Prefix Sum Excluding **i**

This is another very common convention.

Here:

```
prefix[i] = sum of elements before index i
```

In simple words:

`prefix[i]` contains everything **before i**, so `arr[i]` is not included.

We normally create an extra `0` at the beginning.

For:

```
arr = [2, 4, 6, 8, 10]
```

we get:

```
prefix = [0, 2, 6, 12, 20, 30]
```

Think about what every position means:

```
prefix[0] = 0
```

```
prefix[1] = 2
```

```
prefix[2] = 2 + 4 = 6
```

```
prefix[3] = 2 + 4 + 6 = 12
```

```
prefix[4] = 2 + 4 + 6 + 8 = 20
```

```
prefix[5] = 2 + 4 + 6 + 8 + 10 = 30
```

So:

```
prefix[2] = 6
```

because we only take indexes:

```
0 and 1
```

and stop **before index 2**.

Formula

```
prefix[i + 1] = prefix[i] + arr[i]
```

This is one of the cleanest ways to build prefix sums.

Python

```
arr = [2, 4, 6, 8, 10]

prefix = [0] * (len(arr) + 1)

for i in range(len(arr)):
    prefix[i + 1] = prefix[i] + arr[i]

print(prefix)
```

Output:

```
[0, 2, 6, 12, 20, 30]
```

5. Why Do We Sometimes Use `i - 1`?

This is where beginners usually get confused.

Suppose we say:

```
prefix[i] = sum of elements before i
```

Then index `i` itself must not be included.

For example:

```
arr = [2, 4, 6, 8, 10]
```

At:

```
i = 2
```

the elements before index `2` are:

```
index 0 -> 2
index 1 -> 4
```

So:

```
prefix[2] = 2 + 4 = 6
```

That is why we are effectively using:

```
0 ... i-1
```

The important point is:

$i - 1$ is not a rule that all prefix sums must follow.

It is only used when our prefix array is defined as:

```
prefix[i] = sum before i
```

6. Both Conventions Are Correct

These two definitions are both valid.

Include **i**

```
arr    = [2, 4, 6, 8, 10]
prefix = [2, 6, 12, 20, 30]
```

Here:

```
prefix[i] = sum from 0 to i
```

Example:

```
prefix[2] = 2 + 4 + 6 = 12
```

Exclude **i**

```
arr    = [2, 4, 6, 8, 10]  
prefix = [0, 2, 6, 12, 20, 30]
```

Here:

```
prefix[i] = sum from 0 to i-1
```

Example:

```
prefix[2] = 2 + 4 = 6
```

So the important thing is not to memorize:

"Prefix always uses $i - 1$."

Instead, remember:

"First check how the prefix array is defined."

7. Which One Should I Use?

For DSA and interview problems, the **extra-zero version** is often the most convenient:

```
prefix = [0, ...]
```

Why?

Because it makes range-sum calculations very clean.

You do not have to treat the first index as a special case while calculating a range.

The main formula becomes:

```
sum(l, r) = prefix[r + 1] - prefix[l]
```

This is very useful.

8. Range Sum Using Prefix Sum

Suppose:

```
arr = [2, 4, 6, 8, 10]
```

Build:

```
prefix = [0, 2, 6, 12, 20, 30]
```

Now suppose we want:

```
sum from index 1 to 3
```

That means:

```
arr[1] + arr[2] + arr[3]
```

which is:

```
4 + 6 + 8 = 18
```

Using prefix sum:

```
sum(1, 3) = prefix[4] - prefix[1]
```

because:

$$r + 1 = 3 + 1 = 4$$

Therefore:

$$\begin{aligned} &\text{prefix}[4] - \text{prefix}[1] \\ &= 20 - 2 \\ &= 18 \end{aligned}$$

Why does this work?

`prefix[4]` contains:

$$2 + 4 + 6 + 8$$

`prefix[1]` contains:

$$2$$

Subtracting:

$$\begin{aligned} &(2 + 4 + 6 + 8) - 2 \\ &= 4 + 6 + 8 \\ &= 18 \end{aligned}$$

The unwanted part is removed.

9. Another Range Example

Array:

$$\text{arr} = [2, 4, 6, 8, 10]$$

Prefix:

```
prefix = [0, 2, 6, 12, 20, 30]
```

Find the sum from index 2 to 4.

That means:

```
6 + 8 + 10 = 24
```

Formula:

```
prefix[5] - prefix[2]
```

Calculate:

```
30 - 6 = 24
```

So:

```
sum(2, 4) = prefix[5] - prefix[2]
```

10. Why Prefix Sum Is Useful

Without prefix sum, imagine we are asked many times:

```
What is the sum from index 2 to 7?  
What is the sum from index 4 to 9?  
What is the sum from index 1 to 5?  
What is the sum from index 3 to 8?
```

If we calculate every range directly, we may repeatedly visit the same elements.

For one range, that can take:

```
O(n)
```

With a prefix array:

```
sum(l, r) = prefix[r + 1] - prefix[l]
```

The answer takes:

```
O(1)
```

after the prefix array has been built.

Building the prefix array takes:

```
O(n)
```

So for many range queries:

```
Build prefix -> O(n)  
Each query   -> O(1)
```

This can make a huge difference.

11. Time and Space Complexity

For an array of size `n`:

Building prefix sum

```
Time:  O(n)  
Space: O(n)
```

Range query

```
Time: O(1)
```

because we only perform one subtraction.

If the original array is modified in place and extra storage is acceptable to avoid, some implementations can reduce auxiliary space, but the standard prefix-array approach uses $O(n)$ space.

12. Prefix Sum With Negative Numbers

Prefix sum does not require all values to be positive.

Example:

```
arr = [5, -2, 4, -1]
```

Using the extra-zero convention:

```
prefix = [0, 5, 3, 7, 6]
```

Because:

```
prefix[1] = 5  
prefix[2] = 5 + (-2) = 3  
prefix[3] = 5 - 2 + 4 = 7  
prefix[4] = 5 - 2 + 4 - 1 = 6
```

So prefix sum works with:

- positive numbers
 - negative numbers
 - zero
 - mixed values
-

13. A Very Important Pattern: Prefix Sum + Hash Map

Prefix sum becomes even more powerful when combined with a hash map.

A common interview question is:

Find the number of subarrays whose sum is k .

Instead of checking every possible subarray, we can use:

Prefix Sum + Hash Map

The key idea is:

If:

$$\text{prefix}[j] - \text{prefix}[i] = k$$

then:

$$\text{prefix}[i] = \text{prefix}[j] - k$$

So while going through the array, we can ask:

"Have I already seen a prefix sum equal to $\text{current_prefix} - k$?"

If yes, that means a subarray with sum k exists.

This is a very important pattern and appears frequently in DSA.

14. Prefix Sum Example: Subarray Sum Equals K

Suppose:

```
arr = [1, 2, 3]
k = 3
```

Subarrays with sum 3 are:

```
[1, 2]  
[3]
```

So the answer is:

```
2
```

The prefix sums are:

```
1  
3  
6
```

If the current prefix is 6 and:

```
k = 3
```

we look for:

```
6 - 3 = 3
```

We already saw prefix sum 3 .

Therefore, the elements between those two positions have sum 3 .

This pattern is often solved in $O(n)$ using a dictionary.

15. Important Prefix Sum Formulas

Convention 1: Including i

```
prefix[i] = prefix[i - 1] + arr[i]
```

and:

```
prefix[i] = sum(arr[0:i+1])
```

Convention 2: Excluding `i`

```
prefix[i + 1] = prefix[i] + arr[i]
```

and:

```
prefix[i] = sum(arr[0:i])
```

Range Sum

Using the extra-zero convention:

```
sum(l, r) = prefix[r + 1] - prefix[l]
```

16. Common Mistakes

Mistake 1: Mixing the two conventions

For example, building:

```
prefix = [0, 2, 6, 12, 20, 30]
```

but then treating:

```
prefix[2]
```

as:

```
2 + 4 + 6
```

That is wrong for this convention.

Here:

```
prefix[2] = 2 + 4 = 6
```

Mistake 2: Forgetting `r + 1`

With the extra-zero convention:

```
sum(l, r) = prefix[r + 1] - prefix[l]
```

Not:

```
prefix[r] - prefix[l]
```

because `prefix[r]` does not include `arr[r]`.

Mistake 3: Forgetting the extra zero

The extra zero is useful because:

```
prefix[0] = 0
```

Now a range starting at index `0` works naturally.

Example:

```
sum(0, 2)
```

becomes:

```
prefix[3] - prefix[0]
```

If:

```
prefix = [0, 2, 6, 12, 20, 30]
```

then:

```
12 - 0 = 12
```

which is:

```
2 + 4 + 6
```

17. How to Recognize a Prefix Sum Problem

Think about Prefix Sum when the problem says things like:

- Find sum of a range
- Multiple range sum queries
- Sum from index l to r
- Subarray sum
- Count subarrays with sum k
- Find a subarray with a given sum
- Running cumulative sum

Also look for a situation where:

We need the sum of many overlapping parts of an array.

That is a strong signal that Prefix Sum may help.

18. Prefix Sum Mental Model

The easiest way to remember the extra-zero version is:

```
prefix[i] = "How much sum have I collected before reaching index i?"
```

For:

```
arr = [2, 4, 6, 8, 10]
```

think:

```
Before index 0 -> 0
Before index 1 -> 2
Before index 2 -> 6
Before index 3 -> 12
Before index 4 -> 20
Before index 5 -> 30
```

So:

```
prefix[2] = 6
```

means:

```
"Before reaching index 2, I have collected 2 + 4 = 6."
```

This mental model makes the `i - 1` idea much easier to understand.

19. Prefix Sum vs Normal Running Sum

A normal running sum is:

```
total = 0

for x in arr:
    total += x
```

This gives the final total.

Prefix Sum stores **every intermediate total**:

```
arr:      2   4   6   8   10
prefix:  0   2   6  12  20  30
```

So we can reuse those stored totals later.

20. Quick Revision

Prefix Sum Including **i**

```
prefix[i] = sum from 0 to i
```

Example:

```
prefix[2] = 2 + 4 + 6 = 12
```

Prefix Sum Excluding **i**

```
prefix[i] = sum from 0 to i-1
```

Example:

```
prefix[2] = 2 + 4 = 6
```

Recommended DSA Convention

Use:

```
prefix = [0]
```

and:

```
prefix[i + 1] = prefix[i] + arr[i]
```

Then:

```
sum(l, r) = prefix[r + 1] - prefix[l]
```

Complexity

```
Build prefix: O(n)
```

```
Range query:  O(1)
```

```
Space:        O(n)
```

21. Final Thing to Remember

Do not memorize:

```
"Prefix always means i-1."
```

Instead remember:

```
There are different conventions.
```

```
If prefix[i] includes arr[i]:  
    prefix[i] = sum from 0 to i
```

```
If prefix[i] is the sum before i:  
    prefix[i] = sum from 0 to i-1
```

For interview-style range queries, the extra-zero version is usually the easiest:

```
prefix = [0, ...]
```

and:

```
sum(l, r) = prefix[r + 1] - prefix[l]
```

Once this becomes comfortable, the next important Prefix Sum patterns to learn are:

1. Prefix Sum + Hash Map
2. Subarray Sum = K
3. Count Subarrays
4. Prefix Sum with 2D arrays
5. Prefix/Suffix combinations